

I dati di partenza

```
righe = ["V-101;OK", "P-202;K0", "V-103;OK",  
        "T-301;K0", "P-204;OK"]
```

Cosa devi fare

1. Scrivi una funzione `codici_ko(righe)` che, per ogni riga, separi codice ed esito con `split` e restituisca la lista dei soli codici con esito `K0`.
2. Stampa la lista ottenuta.
3. Stampa poi gli stessi codici uniti in una sola stringa con `join`, separati da " / ".

Risultato atteso

```
['P-202', 'T-301']  
P-202 / T-301
```

Suggerimenti di scrittura

- `"P-202;K0".split(";")` restituisce la lista `['P-202', 'K0']`: il codice è l'elemento `[0]`, l'esito l'elemento `[1]`.
- Dai un nome ai pezzi (`codice = pezzi[0]`, `esito = pezzi[1]`): il codice diventa leggibile e il confronto `esito == "K0"` si capisce da solo.
- Schema noto: lista vuota dentro la funzione, ciclo, `append` solo quando la condizione è vera, `return` dopo il ciclo.
- `join` si chiama **sul separatore**, passando la lista: `" / ".join(lista)`.
- Se qualcosa non torna, stampa `pezzi` dentro il ciclo per vedere come `split` ha tagliato la riga.

Prima di guardare pagina 2: scrivi il codice, lancialo e confronta l'output con il risultato atteso. Se non torna, aggiungi dei `print()` di controllo per vedere cosa contengono le variabili.

Listato completo — esercizio_D.py

```
righe = ["V-101;OK", "P-202;K0", "V-103;OK",
        "T-301;K0", "P-204;OK"]

def codici_ko(righe):
    risultato = []
    for riga in righe:
        pezzi = riga.split(";")    # es. ['P-202', 'K0']
        codice = pezzi[0]
        esito = pezzi[1]
        if esito == "K0":
            risultato.append(codice)
    return risultato

ko = codici_ko(righe)
print(ko)
print(" / ".join(ko))
```

Output del programma

```
['P-202', 'T-301']
P-202 / T-301
```

Punti chiave

- `split` trasforma una stringa in una lista; `join` fa il percorso inverso. Insieme servono ogni volta che i dati arrivano come testo con un separatore (CSV, export da software, registri).
- Il separatore di `join` lo scegli tu, spazi compresi: " / " produce un testo più leggibile di "/".

Errore da evitare: non usare `list` come nome di variabile per il risultato di `split`: sovrascrive la funzione `list()` di Python per il resto dello script.